EduPlat Backend

Course Lifecycle Security & Integrity Audit

Read-only review of the repository implementation from course authoring through student completion, final assessment, and downstream effects.

FINAL VERDICT: NOT READY FOR PRODUCTION

Overall answer: NO. The complete lifecycle is not correctly implemented, consistently data-backed, or sufficiently protected from unauthorized access and abuse. Key blockers include authorization gaps, unenrolled quiz access, trusted enrollment cache, broken review flow, inconsistent course duration, missing video progress API, and accepting assessment submission after expiry.

Scope and limits. This is a source-code audit only. No files were modified and no tests were run. The repository contains one explicit SQL migration for quiz duration; SQL Server is configured with Sequelize *synchronize: true*. The deployed database schema and migration history cannot be confirmed from source alone.

Repository: D:/FullStackProjects/EduPlat/server | Audit date: 08 October 2026

1. Lifecycle map

The actual implementation has the following flow. A transition marked NOT IMPLEMENTED has no reachable route in the inspected controllers.

Stage	Actual route / code path	Auth, state, storage	Finding
Draft course	POST /api/course/draft -> CourseService.saveAsDraft	INSTRUCTOR; id from JWT; courses + metadata rows in SQL transaction	Draft creation implemented
Price / metadata update	PATCH /api/course/:id/price -> addPrice	INSTRUCTOR; SQL update by course ID	No ownership check or price DTO; general course update absent
Sections / lessons	SectionService.create exists but no route; POST /api/course/:courseId/section/:sectionId creates lesson	INSTRUCTOR route; lesson and lesson count in transaction	Section API NOT IMPLEMENTED; lesson ownership and ID chain not checked
Video / upload	POST /api/mux/upload-url/:lessonId; signed Mux upload; video.asset.ready webhook stores video	Upload route unguarded; webhook signature verified	No instructor authorization; duplicate webhook is not idempotent
Quiz / questions	POST /api/quiz creates quiz and nested questions/answers	No auth guard; SQL transaction	Unrestricted quiz authoring; quiz start does not require enrollment
Final assessment	POST /api/final/course/:courseId creates final and questions	INSTRUCTOR role only; SQL transaction	Course ownership not checked
Review / publish	POST send-for-review; admin PATCH status	Instructor ownership check on review; admin role for status	Validation query omits sections; method throws after status update
Discovery / enrollment	Published catalog; Stripe webhook -> event -> enrollment listener; Redis enrollment cache	Enrollment unique key; Redis key TTL 7d	Payment, enrollment and cache are not atomic; cache can authorize access
Lesson/video access	GET course/section/lesson; UserEnrolled guard; Mux signed playback URL	JWT + enrollment cache/SQL fallback	Section metadata route has no enrollment or course-status check
Video progress	VideoHandler -> VideoRepository upsert lesson_users	No controller route found	Progress endpoint NOT IMPLEMENTED; internal handler trusts progress
Quiz attempt / answers	POST assessment/start; PATCH answer; POST submit	User JWT; attempt ownership; SQL transaction/row locks	Quiz start bypasses enrollment; post-expiry submit completes normally
Final eligibility	startAssessment(FINAL) -> assertFinalEligible -> counts videos and quizzes	Checks enrollment and content completion	Does not mean course is passed/completed; does not enforce published state
Rewards / certificate / dashboard	Grading events -> listeners/queues; progress upsert; dashboard query	In-memory events and SQL rows	Final event path skips badge listener; completion definitions disagree

Redis and queues: No assessment answers, attempts, timers, locks, or rooms are stored in Redis. The course-related Redis key is enrolled:<userId>:<courseId>, TTL seven days. Assessment expiry uses BullMQ jobId expired:<attemptId> and a 60-second SQL recovery scan.

2. Critical findings

[CRITICAL] Enrollment cache grants access without durable enrollment

Location: payment.service.ts:204; common/guard/lesson.guard.ts:24; course/services/course.service.ts:275
Stripe webhook emits an async enrollment event and independently sets enrolled::=true. Lesson and preview authorization accept a cache hit without SQL verification. If enrollment persistence fails or is delayed, cache grants access for seven days. Make committed enrollment authoritative and update/invalidate cache only after durable state.

[CRITICAL] Quiz start exposes questions to users without enrollment

Location: assessment2/assessment.controller.ts:32; assessment2/assessment.service.ts:50
startAssessment checks final eligibility only for FINAL. QUIZ loads config and questions by assessment ID without checking course enrollment. A logged-in student can read all quiz question and answer text. Enforce course-chain authorization for quiz start and resume.

[CRITICAL] Mux upload and asset management routes are unauthenticated

Location: mux/mux.controller.ts:26; mux/mux.service.ts:23
Upload URL creation, asset retrieval, and deletion have no guards or instructor ownership checks. Any caller can request an upload for a known lesson or invoke asset operations with a known ID. Authenticate and authorize the owning course; restrict asset operations.

[HIGH] Resource ownership missing for course authoring writes

Location: course/course.controller.ts:34; lesson/lesson.controller.ts:31; assessment2/quiz/quiz.controller.ts:20; assessment2/final/final.controller.ts:11
Price change has no owner check; lesson create checks neither course owner nor section/course relationship; quiz creation has no auth; final creation checks role but not course owner. Enforce resource-level ownership using JWT identity and the complete hierarchy.

[HIGH] Video completion can be forged if internal progress handler is exposed

Location: lesson/dto/video.dto.ts:24; lesson/handlers/video.handler.ts:37; lesson/repo/video.repo.ts:48
Client progress is accepted up to duration and completion is true at 80%. No playback evidence, monotonicity, throttling, or enrollment check exists. No public progress route was found, so the route is NOT IMPLEMENTED; internal code is unsafe as-is.

[HIGH] Course duration has conflicting sources of truth

Location: course/entities/course.entity.ts:159; mux/mux.service.ts:91; course/repo/course.repo.ts:638
total_duration is overwritten with one video duration on each ready webhook, while checkout/index queries sum all videos. Recompute the full sum or consistently derive it; make webhook handling idempotent.

[HIGH] Review / publication path is broken

Location: course/services/course.service.ts:154; course/repo/course.repo.ts:75; course/helper/course.validate.ts:7; course/helper/section.validate.ts:5
Review lookup does not load sections, but validator dereferences course.sections. Also service throws unconditionally after attempting status update. A client can receive failure after a successful state mutation. Load required associations, await validation and return after successful transition.

[HIGH] Submission after expiry is accepted as completed

Location: assessment2/assessment.service.ts:213; assessment2/shared/enum/attempt.ts:1
Expired submit only logs a warning, grades saved answers, and sets COMPLETED. There is no EXPIRED state. Enforce an explicit server-side expired transition and distinguish deadline-valid submit.

[HIGH] Final grading does not enqueue course-completion badge

Location: assessment2/assessment.service.ts:369; gamification/badge/listener.ts:14
Final emits assessmentGraded and returns before attempt.graded; badge listener listens only to attempt.graded. Final pass does not enqueue the expected course-completion badge via this listener. Use a durable idempotent event consumed by every downstream handler.

[HIGH] Stored course completion excludes the final

Location: course/progress/course-progress.service.ts:16; course/repo/course.repo.ts:348
Progress completed means all counted videos and quizzes done. It is stored before final pass and does not reflect final completion. Split content completion, final eligibility, and passed course completion.

[MEDIUM] Section metadata is exposed without enrollment/status check

Location: lesson/lesson.controller.ts:24; lesson/repo/lesson.repo.ts:65
GET /section/:id requires a user role but returns lesson/video/quiz metadata without checking course state or enrollment. Quiz start makes this more serious. Apply explicit preview/publication/enrollment policy.

[MEDIUM] Payment, enrollment, cache and events are not atomic

Location: payment/payment.service.ts:120; course/enrollment/enrollment.service.ts:18
Payment row, event, enrollment row and Redis update are separate. A crash can leave a paid payment without enrollment; a cache hit may still grant access. Use durable idempotent webhook processing and recovery/outbox.

[MEDIUM] Course structure lifecycle is incomplete

Location: course/services/section.service.ts:18; lesson/lesson.controller.ts:31
No section controller, general course update, video delete, or assessment update route found. Existing attempt policy when course structure changes is undefined. These transitions are NOT IMPLEMENTED or only partially implemented.

[MEDIUM] Assessment relational integrity relies on app checks

Location: assessment2/models/MCQ&TF;/question.ts:27; assessment2/models/attempt.ts:16; assessment2/models/MCQ&TF;/student.answer.ts:19
Polymorphic examId/assessmentId have no single FK target; student answer columns allow null by model definition, and the DB does not encode that selected answer belongs to the question and attempt assessment. Add schema constraints/normalized ownership and migration-managed schema.

3. Duration, video and progress integrity

Course duration: Course.total_duration is not authoritative. Mux ready events overwrite it with the duration of one video, while other code calculates a dynamic sum. Values can disagree across course, dashboard, search, and checkout consumers.

Video duration: Mux asset metadata is rounded to seconds and is the strongest source on the webhook path. The Video model stores FLOAT and has no bounds; internal VideoDto allows client-supplied duration, but no public video creation route was found.

Playback: Mux uses signed playback policy and the backend issues a playback token expiring in one hour. The route is reached through enrolled lesson access, but enrollment cache can be stale/over-trusted. Mux upload and asset management endpoints are public.

Progress: No controller exposes save/get progress. This is NOT IMPLEMENTED as a student API. The internal repository accepts any integer in [0,duration] and marks completion at 80%, so a single submitted value can forge completion if that handler becomes reachable.

Course progress formula: The calculation is (completed videos + completed quizzes)/(all videos + all quizzes) x 100. It uses distinct counts and requires enrollment. It includes all videos/quizzes regardless of publication/preview status, returns 0 for a zero-content course, and does not include final pass.

Assessment answers: Correct flags live in SQL mcq_answers_quiz.isCorrect. Start responses include answer text and IDs, not isCorrect. Student selections are in SQL student_answers; no Redis answer storage/hash was found. Answer saves validate the assessment chain and unique (attemptId, questionId) prevents duplicate records.

Final availability: Final start checks enrollment and completion of all counted videos/quizzes. This blocks early start under the present formula, but it does not check course publication; the stored completed course flag still does not require passing the final.

Course delete: Published courses are soft-deleted, then service tries to update status to DELETED. Because paranoid deletion excludes the row from ordinary updates, the intended status update may not apply. Draft/rejected/pending courses are hard-deleted without an explicit surrounding transaction.

4. Data flow and source-of-truth table

Data	Source / validation	Stored / authority	Client can manipulate?	Authorization / transaction / risk
Course duration	Mux duration; no aggregate check	courses.total_duration plus dynamic SQL sum; authority ambiguous	Not through course DTO; webhook updates	Webhook signature; video insert + overwrite transaction. Stored number is one video, not sum. HIGH
Video duration	Mux metadata rounded seconds; no model bounds	videos.duration; Mux is source on webhook path	Internal DTO can accept arbitrary number	Public video creation not found. MEDIUM
Video progress	Client integer; bounded by duration only	lesson_users.progress; client-reported value	Yes, if handler is exposed	No enrollment check in handler; route NOT IMPLEMENTED. HIGH
Video completion	Derived at 80% of client progress	lesson_users.completed	Indirectly forgeable	Not proof of playback; affects eligibility. HIGH
Quiz duration	DTO integer 1-180; SQL check in migration	quizzes.duration	At creation	Quiz POST has no auth; transaction with children. HIGH
Final duration	DTO 30-180; integer model	finals.duration	At creation	Instructor role, no ownership check; transaction. HIGH
Question / correct answer	Nested author DTO; one correct answer checked	mcq_questions_quiz / mcq_answers_quiz.isCorrect	Creator chooses content/key	Correct flag omitted from start response; creation authorization weak. HIGH
Student answer	Authenticated choice; scope, revision, deadline checks	SQL student_answers	Can revise with higher revision until deadline	Attempt owner, row lock, unique attempt/question; late final submit flaw. MEDIUM
Attempt ID / status	Attempt server-created; status server-managed	SQL attempts; unique user/type/assessment	IDs supplied on routes, status not accepted from client	Owner check + locks; no EXPIRED state. MEDIUM
expiresAt	Server start time + DB duration	SQL attempts.expiresAt	Client cannot set	SQL authoritative; BullMQ job and recovery are secondary. MEDIUM
Score	Server answer-key grading	attempts.studentScore	No score input accepted	Submission transaction; event after commit can be lost. LOW/MEDIUM
Course progress	Distinct SQL counts of video/quiz completions	progresses upsert; derived data	Influenced by progress forgery	Enrollment required; no encompassing read/write transaction; excludes final. HIGH
Enrollment	Signed Stripe webhook -> event listener	payments, enrollments, Redis cache	No direct enrollment API	Unique user/course; cross-system writes not atomic; cache can grant access. CRITICAL
Course completion	Content formula and separate final event	No single authoritative durable source	Indirect via forged progress	Badge event path misses final; dashboard may show content complete before passing. HIGH

5. State transitions

Entity	Current state -> action	Next state	Allowed / enforcement	Database enforcement
Course	Draft -> create	Draft	Yes; instructor JWT	Status string validator; no DB transition constraint
Course	Draft/rejected -> review	Pending review	Partially; owner check, broken validation and unconditional exception	No state constraint
Course	Pending review -> approve/reject	Published / Rejected	Admin role; existing rejection branch only updates reason	No state constraint
Course	Published -> delete	Soft-deleted; intended DELETED status	Owner filter; status update may miss paranoid row	Paranoid deletedAt
Enrollment	None -> paid webhook	Complete enrollment	Async event; cache write independent	Unique userId/courseId
Video progress	None/existing -> submit progress	Updated; completed at 80%	NOT IMPLEMENTED as route; internal handler accepts jump	Unique videoId/userId; no range check constraint shown
Attempt	None -> start quiz/final	IN_PROGRESS	Auth; final eligibility only for final	Unique examType/assessmentId/userId
Attempt	IN_PROGRESS -> save answer	IN_PROGRESS	Owner, assessment scope, expiry, row lock, revision	Unique attemptId/questionId
Attempt	IN_PROGRESS -> submit (even after expiry)	COMPLETED	Accepted; after expiry only warning	No DB transition constraint; no EXPIRED state
Attempt	IN_PROGRESS -> expiry worker	COMPLETED	Checks DB expiry/state; same grading path	Retries/recovery recheck state
Course progress	Content all complete -> verify	completed=true	Derived from videos + quizzes, requires enrollment	Unique userId/courseId; final not included
Course completion	Final pass -> persist completion	No coherent persisted transition	NOT IMPLEMENTED as one authoritative transition	Events/listeners are separate and non-durable

Concurrency assessment: Answer save and submit lock the attempt row in a SQL transaction; unique indexes protect attempt and answer duplication. Expiry worker rechecks the SQL status and expiresAt, and a recovery scan runs every 60 seconds. These are meaningful controls, but submit-after-expiry semantics, event loss after commit, and non-atomic payment/enrollment/cache remain unresolved.

6. Redis, BullMQ, WebSocket and failure recovery

Redis: Only course enrollment cache was found. Key: enrolled:<userId>:<courseId>, TTL 7 days. Created by Stripe webhook or successful SQL fallback; read by preview/lesson guards. A positive cache value is trusted without SQL. No attempt, answer, timer, lock or assessment room keys were found. Redis failure falls back to SQL.

BullMQ: Assessment expiry job name expired_time; payload includes assessmentId, attemptId, user and type; deterministic job ID expired:<attemptId>; queue defaults to 3 attempts with exponential backoff and remove-on-complete. Worker delegates to expireAttempt, which checks ownership, status, and DB expiresAt. Recovery scan every 60 seconds handles missed/delayed jobs.

Queue limitations: Expiration is largely idempotent, but no durable outbox connects final SQL commit to events/reward jobs. A process crash after commit can lose downstream effects. Badge/quest workers use async map callbacks without awaiting completion; this can let the worker return before writes finish.

WebSocket: Assessment gateway verifies token at connection, loads attempt for token user, checks active state and expiry, then joins attempt:<id>. Socket answer/submit uses server-stored identity and attempt. Client-selected room IDs do not determine the joined room. Namespace CORS is wildcard, but ownership checks are the main boundary.

Failure/recovery: Redis outage falls back to SQL for cache misses; SQL transaction protects attempt state; expired attempts recover through interval scan. Payment/enrollment event processing and Mux ready-webhook idempotency are incomplete. SQL Server outage aborts SQL writes. MongoDB has no observed course-lifecycle role. In-memory grading events can be lost after commit.

Database schema: Unique indexes exist for enrollment, video progress, course progress, attempt per assessment/user/type, student answer per attempt/question, one video per lesson, one quiz per lesson, and one final per course. Polymorphic question/attempt assessment IDs lack direct FK target; no migration set was found for most models, while runtime synchronization is enabled.

7. Abuse scenarios

#	Scenario	Result	Evidence / reason
1	Student sends progress=100 immediately	PARTIALLY	No public route; internal handler accepts valid progress jump and derives completion at 80%.
2	Marks every video complete without watching	PARTIALLY	No route found; internal implementation has no playback evidence.
3	Starts FINAL before completing course	NO (under current formula)	Final start checks enrollment and all counted video/quiz completions.
4	Submits answers after expiration	NO for answer save; YES for submit	saveAnswer rejects expired; submit logs warning then finalizes completed.
5	Submits another user attemptId	NO	REST and WebSocket resolve attempt by authenticated user.
6	Submits another course questionId	NO through save endpoint	Question/answer must belong to attempt assessment/type.
7	Changes answer after submission	NO	Attempt must remain IN_PROGRESS.
8	Duplicate answers concurrently	PARTIALLY	Row lock + unique attempt/question prevents duplicate rows; revision permits legitimate replacement.
9	Starts same assessment in two tabs	NO second attempt	Unique attempt index; existing attempt reused; deterministic expiry job.
10	Repeated reconnect to bypass timeout	NO	Reconnect does not change SQL expiresAt; WebSocket rechecks expiry.
11	Manipulates client clock	NO	Server time and SQL expiresAt are used.
12	Changes courseId/userId in body	PARTIALLY	Core user identity comes from JWT, but instructor writes accept foreign resource IDs without owner checks.
13	Accesses unpublished content	YES for section metadata; quiz start lacks enrollment	Section route has no course status/enrollment test; quiz start by ID lacks enrollment.
14	Accesses deleted content	PARTIALLY	Paranoid lookup normally hides it; intended DELETED status update may not apply post-soft-delete.
15	Instructor modifies active questions	NOT FOUND as update route	No assessment update endpoint found; no explicit snapshot/freeze policy.
16	Two workers expire same attempt	MOSTLY SAFE	SQL state recheck and row lock serialize finalization.
17	Submit races expiration	PARTIALLY	Row lock serializes; request arriving after expiresAt can still complete as normal.
18	Redis disappears	MOSTLY SAFE for misses	Guard falls back to SQL; positive stale cache is unsafe while present.
19	DB fails halfway through finalization	MOSTLY SAFE	Assessment finalization transaction should roll back.
20	Queue retries after partial success	MOSTLY SAFE for expiry	Worker rechecks SQL state; post-commit event side effects remain non-durable.

8. Security rating

Area	Rating	Evidence
A. Course lifecycle correctness	UNSAFE	Broken review path; missing section/progress/update routes; completion mismatch.
B. Course duration integrity	UNSAFE	Stored field is overwritten by one video; other consumers sum.
C. Video duration integrity	PARTIALLY SAFE	Mux metadata on webhook path; upload ownership/model bounds missing.
D. Video progress security	UNSAFE	No student route; internal code trusts client progress and permits 80% jump.
E. Quiz security	UNSAFE	Quiz start lacks enrollment; create route unguarded.
F. Final security	PARTIALLY SAFE	Eligibility check exists; owner and expiry/completion semantics flawed.
G. Question/answer security	PARTIALLY SAFE	Correct flag not returned; answer scope checked; DB polymorphic links weak.
H. Assessment state machine	PARTIALLY SAFE	Locks/transactions exist; no expired state, late submit completes normally.
I. Redis safety	UNSAFE	Enrollment cache is trusted for authorization.
J. Queue safety	PARTIALLY SAFE	Expiry worker rechecks SQL and recovery scans; event side effects not durable.
K. Database integrity	PARTIALLY SAFE	Useful unique indexes; polymorphic links and deployment migration parity uncertain.
L. Authorization	UNSAFE	Multiple ownership gaps, public Mux and quiz creation.
M. Input validation	PARTIALLY SAFE	Global DTO validation; price and semantic/resource checks are missing.
N. WebSocket security	PARTIALLY SAFE	JWT and attempt ownership checked; wildcard CORS and gateway assumptions remain.
O. Transaction integrity	PARTIALLY SAFE	Assessment transactional; payment/enrollment/events/cache are not.
P. Race resistance	PARTIALLY SAFE	Attempt locks/unique constraints help; webhook/event idempotency incomplete.
Q. Data confidentiality	PARTIALLY SAFE	Correct key omitted; questions still exposed through unenrolled quiz start.
R. Failure recovery	PARTIALLY SAFE	Expiry recovery exists; payment, Mux and completion recovery incomplete.

9. Final verdict

Question	Answer
Is the complete Course Lifecycle secure?	NO
Can students manipulate course progress?	PARTIALLY
Can students manipulate video completion?	PARTIALLY
Can students manipulate assessment answers?	PARTIALLY
Can students access another user’s attempt?	NO
Can students start the Final before course completion?	NO, under current content formula
Can students submit after expiration?	YES
Can race conditions corrupt assessment state?	PARTIALLY
Is the database the correct source of truth?	PARTIALLY
Is Redis used safely?	NO
Are queue jobs safe and idempotent?	PARTIALLY
Are course/video/quiz/final durations trustworthy?	NO
Overall production readiness	NOT READY

10. Top 10 fixes by priority

1. Stop treating Redis enrollment cache values as authorization; make committed SQL enrollment authoritative.
2. Add enrollment and course-resource authorization to every quiz start, lesson read, video URL, and assessment route.
3. Add instructor ownership checks to price changes, lesson creation, final creation, and Mux upload/asset operations; guard quiz creation.
4. Make video progress server-verifiable; do not treat client-reported duration as proof of watching.
5. Correct review validation and return semantics so publication loads actual sections and reports committed state accurately.
6. Define one course-duration source of truth and recompute the full duration idempotently when video assets change.
7. Add an explicit expired attempt state and reject or consistently finalize submissions after the deadline.
8. Separate content completion, final eligibility, and passed-course completion; persist course completion only after final pass.
9. Use durable, idempotent event delivery/outbox for payment enrollment and final rewards/certificates.
10. Add migration-managed constraints and idempotency for assessment ownership, Mux webhook events, payment sessions, and dependent records.

Audit evidence locations

The principal implementation files referenced in this report are under src/course, src/lesson, src/assessment2, src/mux, src/payment, src/redis, src/queue, and src/gamification. File paths are shown inline in the findings.

End of report